Security is harder. A data centre is controlled. An edge device sits in a public space, where somebody could literally walk up and plug in a USB stick.

These constraints shape every architectural choice. Tools that work in the cloud don't always translate, which is why a whole ecosystem of edge-specific Kubernetes distributions has grown around the standard one.

The lightweight distributions, briefly

Standard Kubernetes was never designed for small devices. The control plane alone wants a couple of gigs of RAM. On a Raspberry Pi, you'll feel every megabyte.

K3s is the one most teams reach for first. It's a certified Kubernetes distribution from Rancher, now part of SUSE. The binary is under seventy megabytes. It swaps etcd for SQLite by default. The whole control plane fits in half a gigabyte of RAM. Looks and behaves exactly like Kubernetes, because it is.

MicroK8s from Canonical is another popular choice. Installs as a snap on Ubuntu and has a simple add-on system. The resource footprint is a bit higher than K3s but still manageable.

KubeEdge takes a different approach. Instead of shrinking Kubernetes, it extends it. The control plane stays in the cloud. A lightweight agent called EdgeCore runs on the edge device and syncs state with the cloud. Devices can keep running even when disconnected.

k0s is a single-binary Kubernetes distribution that stays close to upstream Kubernetes while trimming the fat.

For most teams, the choice ends up between K3s and KubeEdge -- K3s if each edge site is a self-contained cluster and KubeEdge if you want a single logical cluster spanning cloud and edge.

Getting K3s running on a small device

The easiest way to understand what we're talking about is to install K3s somewhere small.

```
curl -sfL https://get.k3s.io | sh -
```

That's it. One command. By the time it finishes, you have a working single-node Kubernetes cluster.

```
sudo k3s kubectl get nodes
sudo k3s kubectl get pods -A
```

You'll see a node in *Ready* state and system pods running. The whole thing is probably using less than five hundred megabytes of RAM.

To add workers, run a different install command on each, pointing at the main server.

```
curl -sfL https://get.k3s.io | K3S_
URL=https://main-server:6443 \
  K3S_TOKEN=mynodetoken sh -
```

Within a minute, the new node shows up in the cluster. From there, it behaves exactly like standard Kubernetes. kubectl works. Helm works. The first time I did this on a Raspberry Pi 4, I was genuinely surprised at how smooth it was.

Designing for intermittent connectivity

This is where edge architecture diverges from cloud. In a data centre, you assume the control plane is always reachable. At the edge, you cannot. A few patterns show up in practice.

Local autonomy: Each edge site runs its own cluster. The cloud is used for management, config sync, and metrics aggregation. If the cloud link goes down, the edge keeps running. This is the K3s pattern.

Hybrid control plane: The control plane lives in the cloud, but edge nodes cache state locally. KubeEdge works this way.

GitOps-driven sync: Edge clusters pull their configuration from Git. Tools like Flux or Argo CD run on the edge cluster and periodically check for updates. If Git is unreachable, the cluster keeps running the last known good configuration. This pattern scales beautifully to thousands of sites.

For my friend's retail chain, we ended up with K3s plus Argo CD. Each store had its own cluster. All configurations lived in Git. Changes were just commits. An audit was built in.

Workload patterns that fit the edge

Not everything belongs at the edge. A few patterns work especially well.

Latency-sensitive processing: Anything that needs to respond in milliseconds should run close to the data — industrial control, real-time video analytics, point-of-sale validation.

Bandwidth-sensitive filtering: If your sensors generate a gigabyte of raw data per hour but only ten megabytes is interesting, filter at the edge. Huge for computer vision workloads.

Resilience-critical services: Services that must keep running when connectivity drops — the petrol pump that still needs to dispense fuel, the hospital system that still needs to admit patients.

Compliance or data residency: Sometimes data cannot leave a specific location for legal reasons.

What doesn't belong at the edge is anything that would be fine in the cloud -- analytics dashboards, model training, long-term storage. Push these up. Keep the edge focused.

Fleet management, the real daily problem

Running one edge cluster is easy. Running a hundred is where the actual work lives.

Rancher is the heavyweight choice for managing lots of K3s clusters. It offers a central UI, policy management, user access control, and visibility across your fleet.

Fleet, from Rancher, is specifically for GitOps across many clusters. You define bundles of workloads and target